



Universidade de Brasília

Departamento de Ciência da Computação

Arquitetura RISC-V



Instruction Set Architecture - ISA

- A ISA define a interface entre o software e o processador
- Informações típicas:
 - Quais instruções o processador executa
 - Qual o formato destas instruções
 - Que tipos de dados são suportados
 - Quais os registradores estão disponíveis
 - Que modos de endereçamento são utilizados



Principais ISAs

■ ARM

- ARMv7 (32 bits) e ARMv9 (64 bits)

■ x86 (32 bits) e EM64T (64 bits, origem AMD)

■ RiscV

- Desenvolvida na UC Berkeley como uma ISA aberta
 - Projeto iniciado em 2010 por alunos do Patterson
- Ampla gama de aplicações
 - Desde sistemas embarcados até supercomputadores
- Gerida pela Fundação RISC-V <http://www.riscv.org>





Arquiteturas RISC-V

RV32 – registradores de 32 bits

RV64 – registradores de 64 bits

RV128 – registradores de 128 bits

Tipo de instruções	Sufixo
ISA de inteiros	I
Instruções de Multiplicação e Divisão	M
Instruções atômicas (sincronização de memória)	A
Instruções de ponto flutuante (precisão simples)	F
Instruções de ponto flutuante (precisão dupla)	D
ISA Geral	IMAFD = G
Conjunto reduzido para sistemas embarcados	E

Modo normal: Instruções com 32 bits de tamanho

Modo condensado: Instruções com 16 bits de tamanho

Modo expandido: Instruções com $n \times 16$ bits de tamanho (48,64,96,...)



Operandos da RiscV RV32

Local do operando	Exemplo	Comentários
Banco de 32 Registradores	$x0, x1, x2, \dots, x31$	Local de acesso mais rápido a variáveis
Memória RAM	$M[0], M[8], M[16], \dots, M[2^N-8]$ $M[0], M[4], M[8] \dots, M[2^N-4]$ $M[0], M[2], M[4] \dots, M[2^N-2]$ $M[0], M[1], M[2] \dots, M[2^N-1]$	<i>double word (RV64)</i> <i>word,</i> <i>half-word</i> <i>byte</i>
Acesso imediato	<code>addi x5, x5, 123</code>	Local de acesso mais rápido a constantes



Instruções Aritméticas

- Soma e subtração: add e sub
- Três operandos
 - add a, b, c $\Rightarrow$ $a = b + c$
 - sub a, b, c $\Rightarrow$ $a = b - c$
- Todas operações aritméticas seguem este padrão
- Simplicidade favorece a regularidade
 - Regularidade simplifica a implementação
 - Simplicidade leva a alto desempenho com custos reduzidos



Exemplo Aritmético

■ Código C:

$$f = (g + h) - (i + j)$$

■ Código RiscV:

```
add t0, g, h           // t0 - temporário
add t1, i, j           // t1 - temporário
sub f, t0, t1          // f = t0 - t1
```

Variáveis => registradores !



Registadores RV

Registrador	Nome	Descrição	Preservado?
x0	zero	Constante zero <i>hardwired</i>	-
x1	ra	Endereço de retorno	Sim
x2	sp	<i>Stack Pointer</i>	Sim
x3	gp	<i>Global Pointer</i>	-
x4	tp	Thread Pointer	-
x5	t0	Temporário	Não
x6-7	t1-2	Temporários	Não
x8	s0/fp	Reg Salvos / <i>frame pointer</i>	Sim
x9	s1	Registadores Salvos	Sim
x10-11	a0-1	Argumentos / val retorno	Não
x12-17	a2-7	Argumentos função	Não
x18-27	s2-11	Registadores Salvos	Sim
x28-31	t3-6	Temporários	Não



Reescrevendo o Exemplo

- Considerando a convenção adotada, podemos associar:

□ $f \Rightarrow x19$

□ $g \Rightarrow x20$

□ $h \Rightarrow x21$

□ $i \Rightarrow x22$

□ $j \Rightarrow x23$

$$f = (g + h) - (i + j)$$



```
add    x5, x20, x21
add    x6, x22, x23
sub    x19, x5, x6
```

```
# temporário x5 = g + h
# temporário x6 = i + j
# f = (g + h) - (i + j)
```



Operandos na Memória

- Dados tem que ser transferidos da memória para os registradores
- Instruções de acesso à memória
 - **load** => transfere da memória para o registrador
 - **store** => transfere do registrador para a memória
- Exemplo:
 - `lw t0, 16(s0)`
 - Endereço = $s0 + 16$
 - $t0 \leq \text{mem}[s0 + 16]$
 - `sw t0, 16(s0)`
 - Endereço = $s0 + 16$
 - $\text{mem}[s0 + 16] \leq t0$



Registradores x Memória

- Registradores tem acesso mais **rápido** do que a memória
 - A memória é muito maior que o banco de registradores
- Operandos na memória requerem operações de *load* e *store* para transferi-los para os registradores
 - Mais instruções são executadas
- O compilador (e o programador) deve usar preferencialmente os registradores para armazenar as variáveis
 - Otimizar o tempo de execução
 - Usar memória apenas quando for muito necessário



Operando imediato

- Operando é uma constante dentro da instrução:
 - `addi t0, t1, 27`
 - Instruções terminadas em “i” utilizam imediato
- É um exemplo de simplificação de projeto:
 - Uso de pequenas constantes é frequente
 - Colocar a constante dentro da instrução evita acesso à memória ou a registrador



Organização da Memória

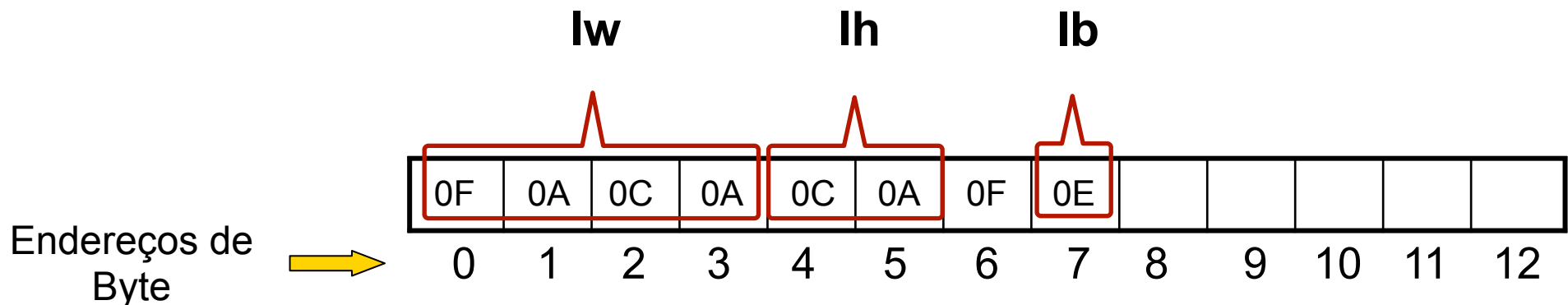
- Vista como um grande array unidimensional, com endereços sequenciais, começando em 0
- Um endereço de memória é um índice no array.
- "Byte addressing" significa que o índice aponta para um byte na memória.
- Tipos de dados:
 - Double (**ld**): 64 bits → 8 endereços
 - Word (**lw**): 32 bits → 4 endereços
 - Half Word (**lh**): 16 bits → 2 endereços
 - Byte (**lb**): 08 bits → 1 endereço

Endereço	Dado
0	8 bits
1	8 bits
2	8 bits
3	8 bits
4	8 bits
5	8 bits
6	8 bits
7	8 bits
8	8 bits
9	8 bits
...	...
2^N-1	8 bits



Organização da Memória

- Cada instrução agrupa um conjunto de bytes:
 - ld: endereços múltiplos de 8 (0, 8, 16, 24, ...)
 - lw: endereços múltiplos de 4 (0, 4, 8, 16, ...)
 - lh: endereços múltiplos de 2 (0, 2, 4, 6, ...)
 - lb: endereços múltiplos de 1 (0, 1, 2, 3, 4, ...)



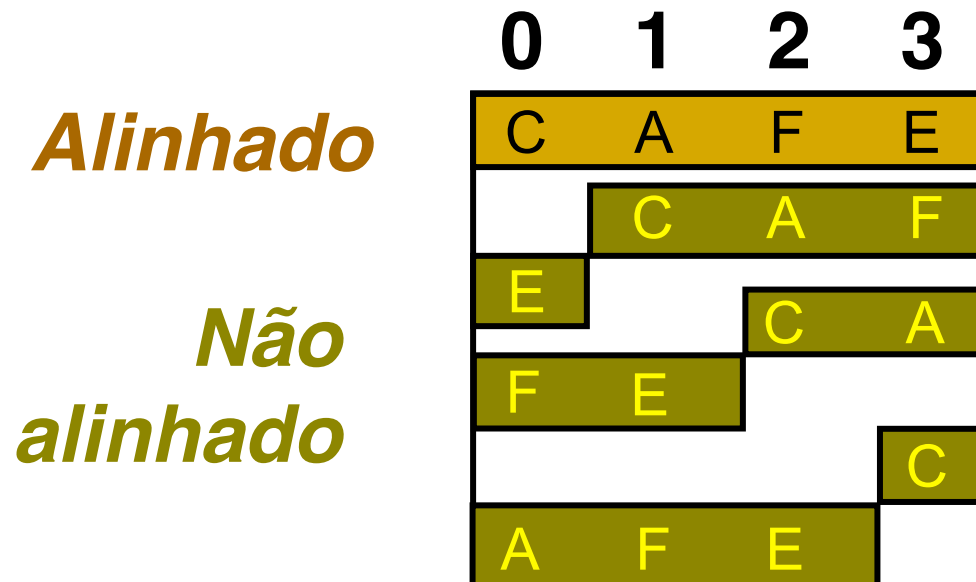


Organização da Memória

2^{32} *bytes* com endereços de *byte* de 0, 1, 2, 3, ... $2^{32}-1$

2^{30} *words* com endereços de *byte* de 0, 4, 8, ... $2^{32}-4$

Words são **alinhadas**, i.e., quais são os valores dos 2 bits menos significativos do endereço de uma *word*?





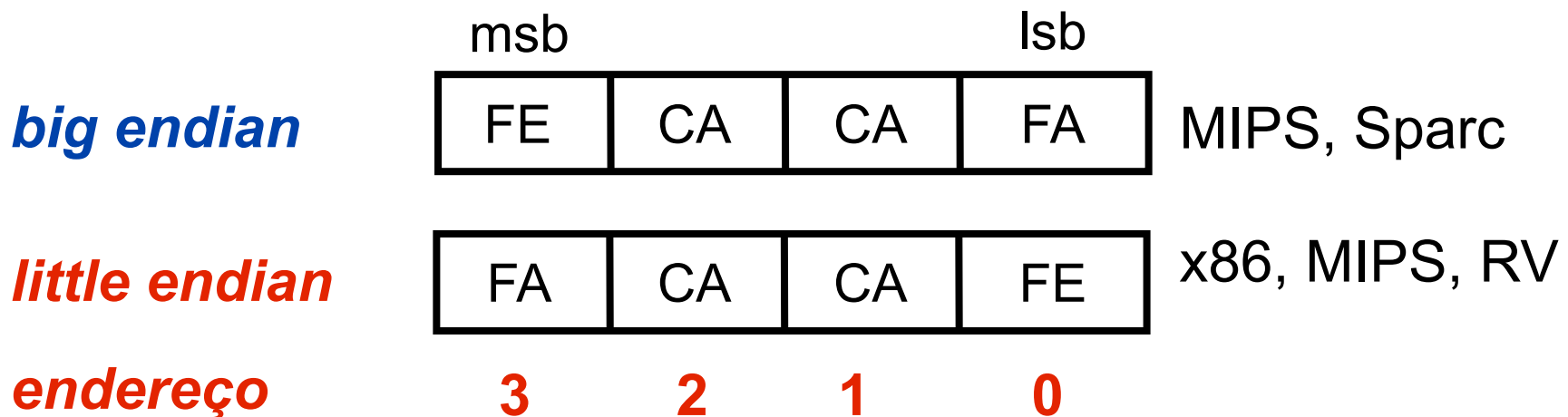
Ordenamento dos *Bytes*

- Existem duas formas de ordenar os bytes em uma palavra.

- Registradores:

- $t0 \leq 0xFACACAFE$

- Na memória:





Exemplo

- Um vetor de inteiros A tem 100 posições. O compilador associou a variável h ao registrador x21. Temos ainda que o endereço base do vetor A é dado em x22. Qual o código para:

$A[12] = h + A[8] ?$

- A nona posição do vetor A, $A[8]$, está no offset $8 \times 4 = 32$

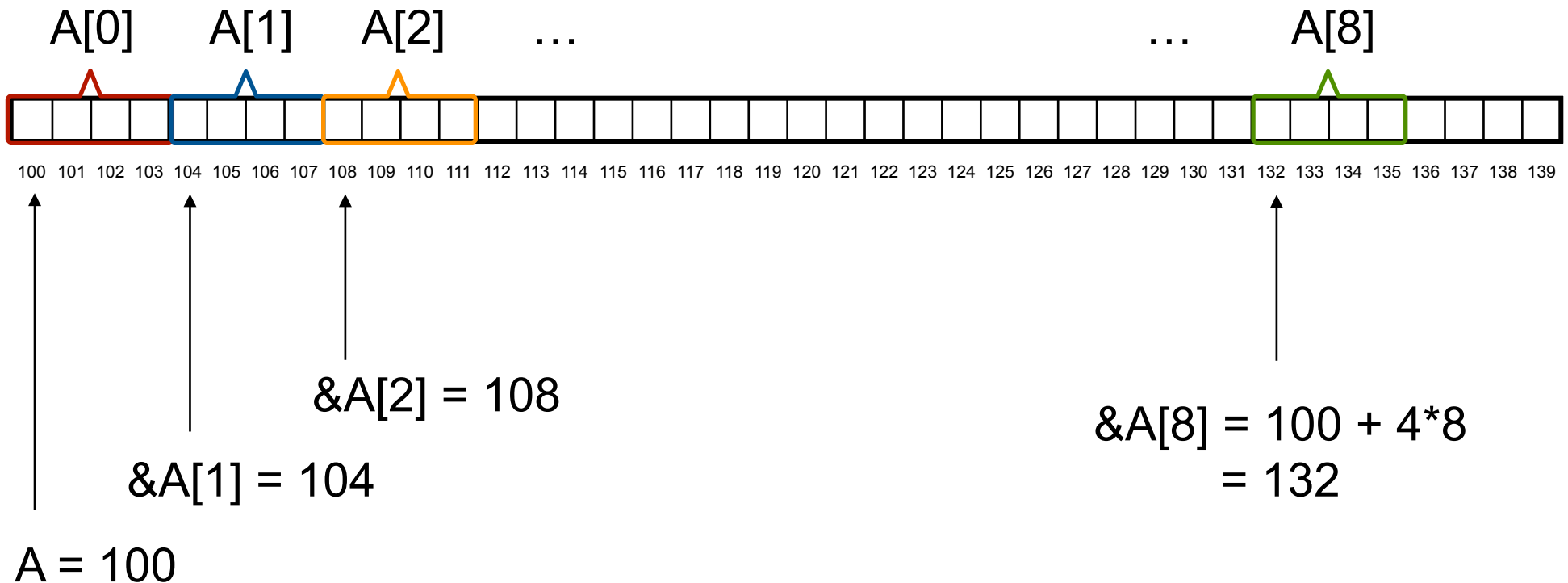
```
lw x9, 32(x22)      # temporário x9 = A[8]
add x9, x21, x9      # temporário x9 = h + A[8]
```

- A décima-terceira posição do vetor A, $A[12]$, está no offset $12 \times 4 = 48$

```
sw x9, 48(x22)      # armazena x9 em A[12]
```



Vetor na Memória





Exercício

- Temos ainda que o endereço base do vetor A é dado em x22, e que as variáveis i e g são dadas em x20 e x21, respectivamente. Qual o código para

$$A[i+g] = g + A[i] - A[0] ?$$